



THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA

CREATE CHANGE

Advanced Database Systems (INFS3200)

Lecture 4: Distributed Transaction Management

Lecturer: Dr Junliang Yu

School of Electrical Engineering and Computer Science (EECS)

Faculty of Engineering, Architecture and Information Technology

The University of Queensland

Don't miss out in March & April

EAIT Employability has a line-up of events for IT, Electrical Engineering, and Computer Science students



Workshop: What is employability?

19 March, 12:30–1:30 pm



Career Conversations: Open the door to your future at Workday

25 March, 12:30–1:30 pm



Google's Tech Internship information session

28 March, 11 am – 12:30 pm



Industry Masterclass: SAS Institute on data and analytics

7 April, 12:30–1:50 pm



Workshop: Getting your resume ready

8 April, 12:30–1:30 pm

Sign up for these
events at

[eait.uq.edu.au/
employability-events](http://eait.uq.edu.au/employability-events)



Topics

Transaction Management in centralized DBMS

- Properties of database transactions
- Concurrency Control
- Durability and Recovery
- Database State

Distributed Transaction Management

- Concurrency Control and Recovery in Distributed Databases
- Two phase Commit Protocol
- Maintaining Replicas in Distributed Databases

Examples of Transaction Processing Systems (TPS)

Airline reservation

Online Retail sales

Course enrollment

Stock exchange

What's important there?

Transaction Processing Systems are systems designed to manage and execute database transactions, often involving large databases with multiple concurrent users.

TPS are usually built upon DMBS

DBMS serves as the foundation for TPS by providing essential data storage, retrieval, and management functions. TPS leverages the DBMS to execute transactions securely and efficiently, ensuring data consistency, integrity, and persistence.

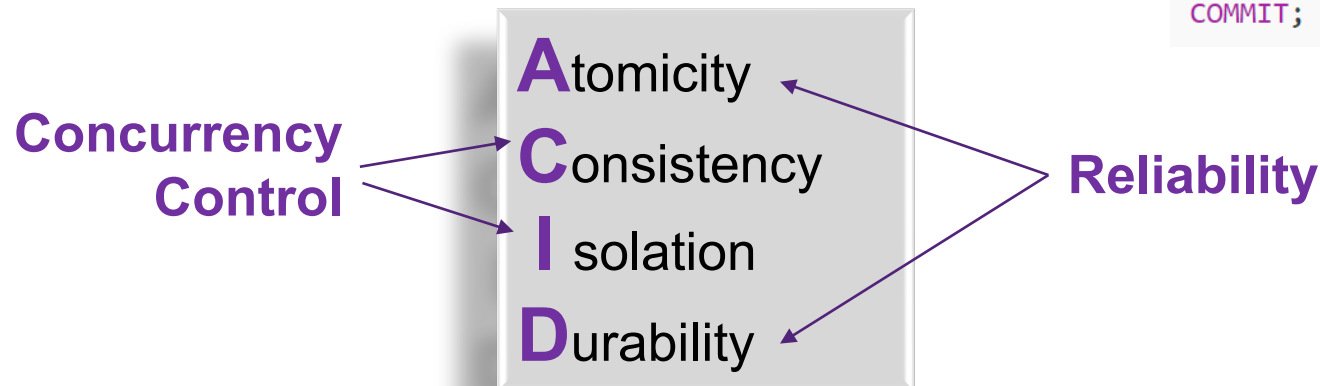
Transactions: Basic Concepts

Database Transaction

- A **logical unit** of database processing
- A **sequence of read/write** operations treated as one atomic unit

Transaction Processing Systems

- designed to manage and execute database transactions, often involving large databases with multiple concurrent users.
- Guaranteed **ACID** properties even in the presence of **conflicting operations**



Example: A transaction transferring \$500 from Account A to Account B.

```
BEGIN TRANSACTION;

-- Step 1: Debit Account A
UPDATE Accounts
SET balance = balance - 500
WHERE account_id = 'A';

-- Step 2: Credit Account B
UPDATE Accounts
SET balance = balance + 500
WHERE account_id = 'B';

-- Commit the transaction if all steps succeed
COMMIT;
```

What could go wrong without ACID constraints?

Transactions and Buffers

READ (X)

1. Find the address of the disk block that contains item X
2. Copy the disk block into a buffer in main memory (if not already)
3. Copy item X from buffer to program variable named X

WRITE (X)

1. Find the address of the disk block that contains item X
2. Copy the disk block into a buffer in main memory (if not already)
3. Copy item X from program variable named X into correct location in buffer
4. Store updated disk block from buffer back to disk

Properties of a Transaction

• Atomicity

- A transaction is a sequence of database operations that are either performed entirely or are not performed at all. (described as an “all-or-nothing” proposition)

• Consistency

- A transaction must transform the database from one consistent state to another consistent state.

• Isolation

- Transactions execute independently of one another. The partial effects of incomplete transactions should not be visible to other transactions.

• Durability

- The effects of a committed transaction are permanently recorded and must not be lost because of a subsequent failure.

Initial State (Consistent State)

- Account A: \$500
- Account B: \$300
- Account C: \$100
- Total funds: \$900 ✓

Transaction Failure Example:

T1: Transfer \$200 from A to B →
Account A = \$300
T2: Transfer \$100 from A to C →
Account A = \$200

Result: Account B remains at \$300.

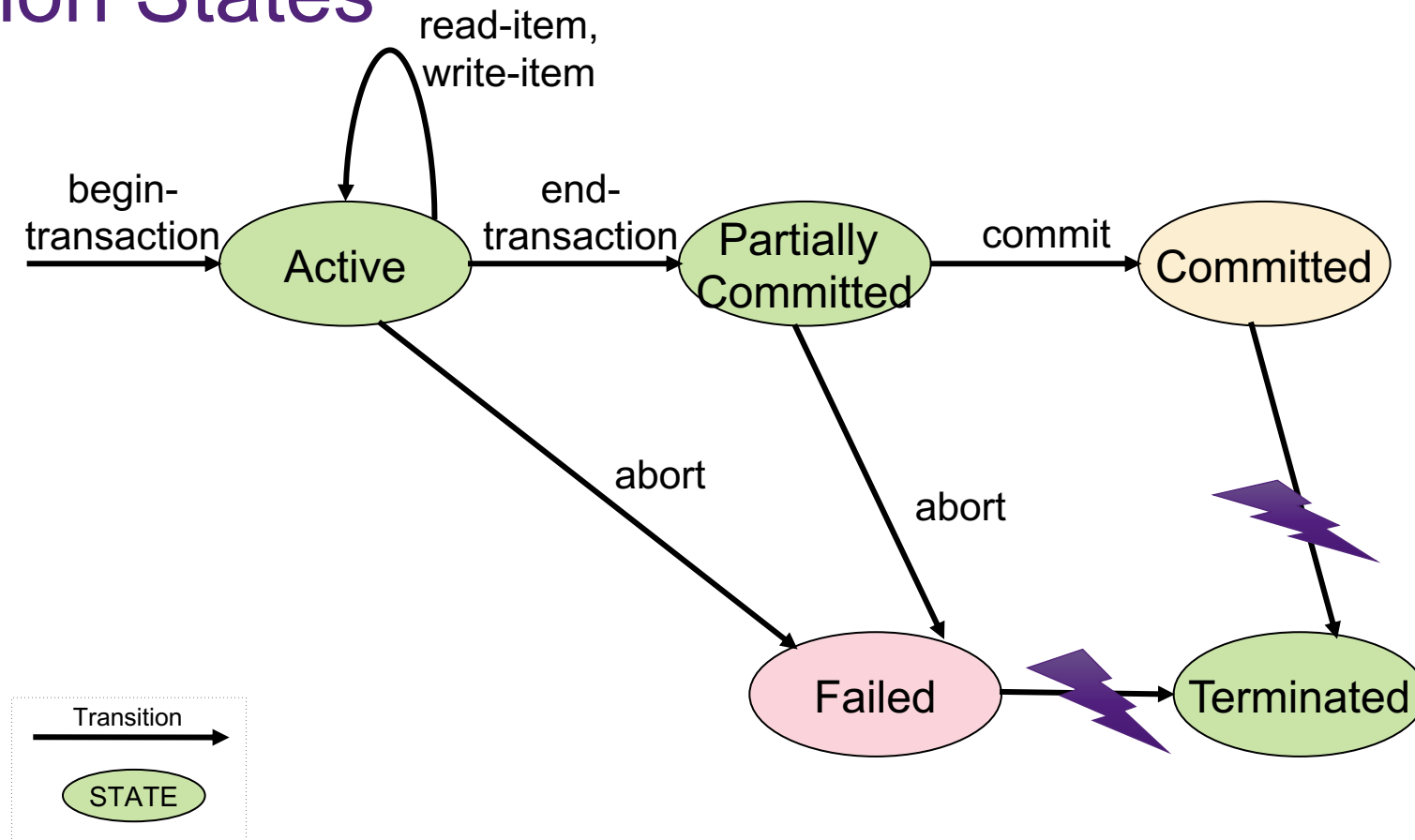
If no concurrency control, T2 would overwrite T1's result on A.

This is called **Lost updates**

Final State (Inconsistent State)

- A: \$400, B: \$500, C: 200
- Total funds: \$1100 ✗ (Inconsistent)

Transaction States



Where may things go wrong?

Commit in Centralized Databases

All database access operations of the transaction have been executed successfully and their effect on the database has been recorded in the log

System log entries:

1. [start-transaction, T];
2. [read-item, T, X];
3. [write-item, T, X , old-value, new-value];
4. [commit, T];

After the commit point, the transaction T is said to be committed

What is Concurrency Control ?

Transaction processing systems are systems designed to manage and execute database transactions, often involving large databases with multiple concurrent users.



Multiple users are **concurrently** executing their own transactions



Every transaction may access different data items, some of transactions could be **accessing the same data item - conflict**



Through multi-programming operating systems, the processes of users are **interleaved**

Schedule

A **schedule** is the order of execution of operations from various transactions

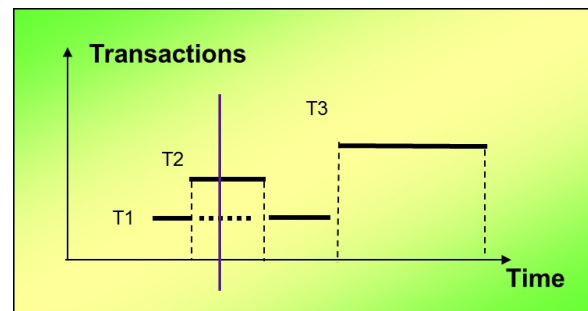
A formal definition of a schedule

- A schedule S of n transactions $T_1, T_2, \dots, T_n$ is an **ordering of the operations** of these transactions, given that $\forall T_i \in S$, **the order of operations** of T_i in S is the same as that in the original T_i

Schedules consider **read-item**, **write-item**, **commit** and **abort** operations only and the order of operations in S is a **total order**.

Interleaved Concurrent Transactions:

At any time point, there may be many active transactions but only one is executing.



T1	T2	T3
R(A)		
	W(A)	
W(A)	COMMIT	
COMMIT		
		W(A)
		COMMIT

Serial Transactions

Question:

Given three transactions T1, T2, T3, how many possible **serials** for them to be executed correctly (i.e., find their correct serial executions)?

Answer: There are six correct serials:

T1→T2→T3

T2→T3→T1

T1→T3→T2

T3→T1→T2

T2→T1→T3

T3→T2→T1

Operations in serial transactions are not interleaved.

Serial Schedules

Schedules that execute each transaction one after another without any interleaving.

- Formally, a schedule S is **serial** if for every transaction T participating in S , all operations of T are executed **consecutively**.
- There are $n!$ serial schedules for n transactions

A **serial schedule is always correct**, but unacceptable in practice!

- Why correct?
- Why unacceptable?

Operations in serial transactions are not interleaved.

Conflicting Operations

Operations of a schedule are in **conflict** if they meet all the following three conditions at the same time:

- they belong to different transactions
- they access the same item X
- at least one is a write-item (X)

A **Write-Write** conflict occurs when two transactions try to write different values to the same item. The final value depends on which write happens last, potentially leading to **lost updates**.

A **Read-Write** conflict arises because one transaction might read outdated data or uncommitted data (leading to **dirty read**).

Given two transactions with the following concurrent operations.

Conflicting operations

Case 1:	r1(X); w2(X)
Case 2:	r1(X); w1(X)
Case 3:	r1(X); r2(X)
Case 4:	w1(X); w2(Y)
Case 5:	w1(X); w2(X)
Case 6:	w1(Y); r2(Y)

Examples of Schedules

Serial Schedule (**non interleaving**)

T1	T2
read-item (X); X:=X-N; write-item (X); read-item (Y); Y:=Y+N; write-item (Y);	 read-item (X); X:=X+M; write-item (X);

T1	T2
read-item (X); X:=X-N; write-item (X); read-item (Y); Y:=Y+N; write-item (Y);	 read-item (X); X:=X+M; write-item (X);

“Non-Serial Schedule”: Schedule with interleaved operations.

Example of Interleaved Transactions

Two transactions have their operations interleaved in such a way that it makes the value of some data item incorrect

Consider an example:

Initially: accounts X has \$500 and Y has \$500.

T1: transfer \$100 from X to Y

T2: deposit \$50 to X

What will be final values of X and Y?

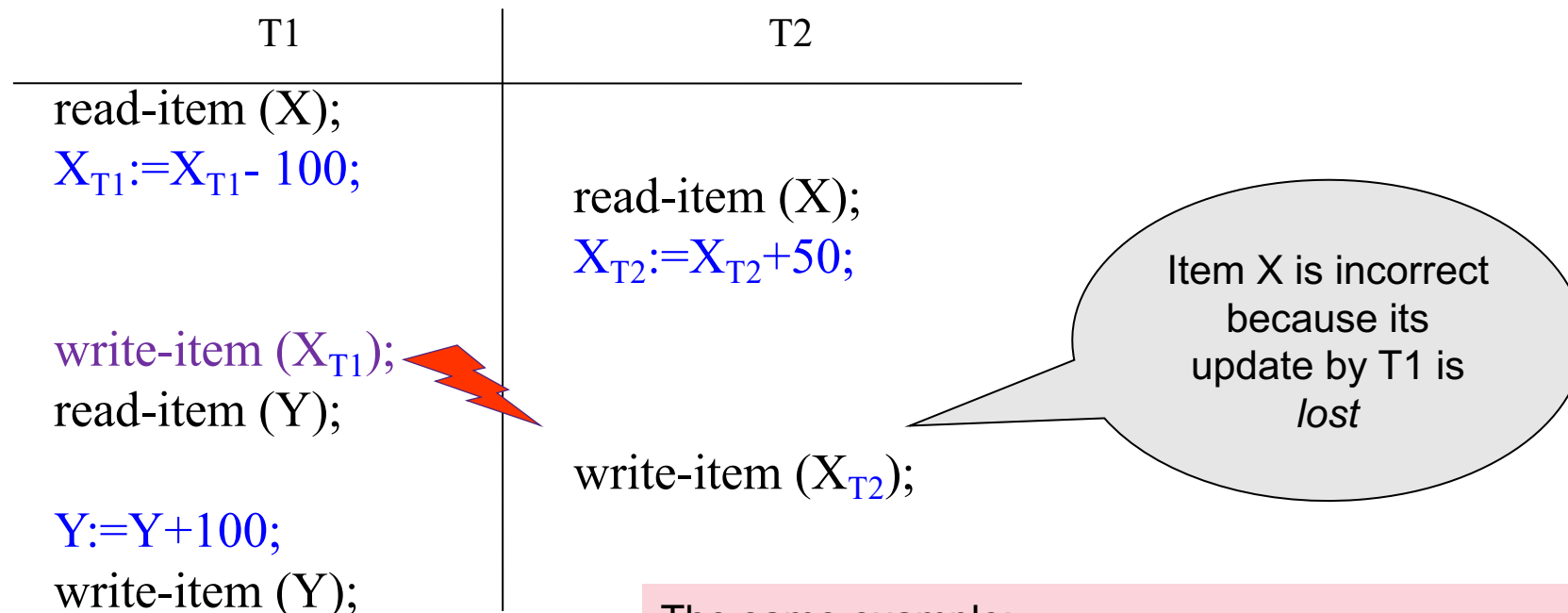
X=450, Y=600

T1	T2
read-item (X); $X_{T1} := X_{T1} - 100;$	
	read-item (X); $X_{T2} := X_{T2} + 50;$
write-item (X_{T1}); read-item (Y);	
$Y := Y + 100;$ write-item (Y);	write-item (X_{T2});

Now incorrectly: X=550, Y=600
(the deduction of \$100 in T1 is lost)

Non-serializable (Lost Update)

Two transactions have their operations interleaved in such a way that it makes the value of some data item incorrect



The same example:
Initially: accounts X has \$500 and Y has \$500.
T1: transfer \$100 from X to Y
T2: deposit \$50 to X

Problems in Concurrency Control

Lost Update Problem

- Update from one transaction is lost

Dirty Read Problem

- Update from one transaction fails when in the meanwhile another transaction has already accessed the updated item

Incorrect Summary Problem

- One transaction is performing an aggregate function on multiple database items while other transaction(s) are updating some of those items

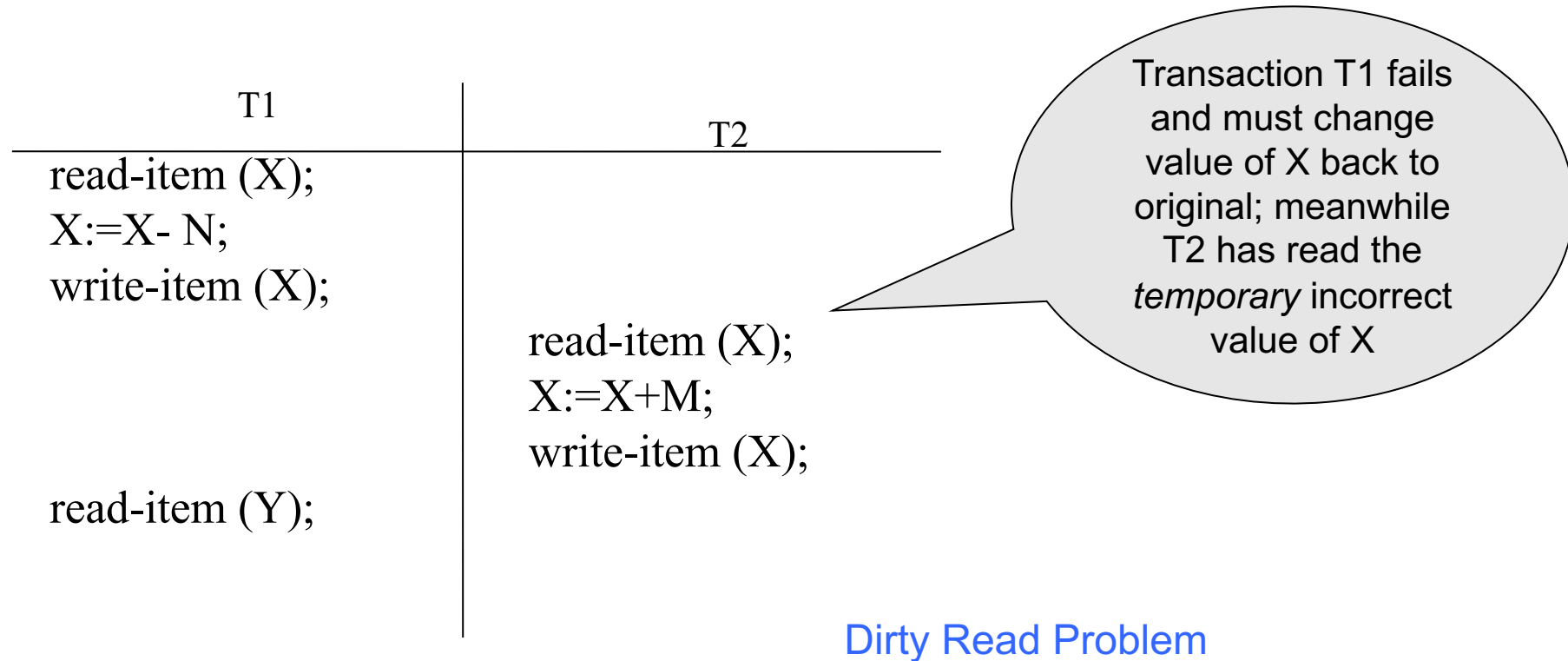
Unrepeatable Read Problem

- A transaction reads an item twice but its value is changed by another transaction in between the two reads

Spot the Concurrency Control Problem

T1	T2
read-item (X); X:=X- N; write-item (X);	read-item (X); X:=X+M; write-item (X);
read-item (Y);	

Spot the Concurrency Control Problem



Concurrency Control based on Locking

Binary Locks (either locked or unlocked)

- When one transaction has a lock on an item X, no other transaction can read or write X
- Generally considered too restrictive for performance of the database system

Read/Write Locks

- When one transaction has a READ lock on an item X, other transactions can also read X (shared lock)
- When one transaction has a WRITE lock on an item X, other transactions can not write X (exclusive lock)

Example

T_1	T_2
<code>read_lock(Y);</code> <code>read_item(Y);</code> <code>unlock(Y);</code> <code>write_lock(X);</code> <code>read_item(X);</code> <code>X := X + Y;</code> <code>write_item(X);</code> <code>unlock(X);</code>	<code>read_lock(X);</code> <code>read_item(X);</code> <code>unlock(X);</code> <code>write_lock(Y);</code> <code>read_item(Y);</code> <code>Y := X + Y;</code> <code>write_item(Y);</code> <code>unlock(Y);</code>

Assume initially $X=20$, $Y=30$.

For Serial (i.e., correct) Schedules:
 If $T_1 \rightarrow T_2$ then $X=50$, $Y=80$;
 If $T_2 \rightarrow T_1$ then $X=70$, $Y=50$.

Non Serializable

T_1	T_2
<code>read_lock(Y);</code> <code>read_item(Y);</code> <code>unlock(Y);</code>	<code>read_lock(X);</code> <code>read_item(X);</code> <code>unlock(X);</code> <code>write_lock(Y);</code> <code>read_item(Y);</code> <code>Y := X + Y;</code> <code>write_item(Y);</code> <code>unlock(Y);</code>
<code>write_lock(X);</code> <code>read_item(X);</code> <code>X := X + Y;</code> <code>write_item(X);</code> <code>unlock(X);</code>	

X and Y **unlocked**
too early!

Non-Serializable means the
results of transactions are
wrong!

wrong results are $X=50$;
 $Y=50$

Two-Phase Locking (2PL)

Two-Phase Locking (2PL) is a concurrency control protocol that ensures **serializable** transaction schedules by regulating when locks are acquired and released.

How Does 2PL Work?

2PL divides each transaction's lock operations into two distinct phases:

1. Growing Phase:

1. A transaction can **acquire locks** (read or write).
2. It **cannot release any locks** during this phase.

2. Shrinking Phase:

1. Once a transaction releases **any lock**, it enters the shrinking phase.
2. In this phase, the transaction can only **release locks**; it **cannot acquire any new locks**.

All locks must be acquired **before the first unlock** to comply with 2PL.

Example of Two-Phase Locking (2PL)

```
T1:
read_lock(Y);
read_item(Y);
write_lock(X);  -- Lock acquired before unlocking
read_item(X);
X := X + Y;
write_item(X);
unlock(Y);      -- First unlock starts shrinking phase
unlock(X);
```

✓ **Follows 2PL:** All locks are acquired before the first unlock.

```
T1:
read_lock(Y);
read_item(Y);
unlock(Y);      -- ✗ Violates 2PL (Unlock occurs before acquiring all locks)
write_lock(X);
read_item(X);
X := X + Y;
write_item(X);
unlock(X);
```

✗ **Violates 2PL:** Unlocking occurs before acquiring all required locks.

Deadlock

A **deadlock** occurs when two or more transactions are waiting indefinitely for each other to release locks. This cyclic dependency causes all involved transactions to be stuck permanently unless external intervention occurs.

Consider two transactions:

Transaction T1:

- Locks X
- Tries to lock Y (but Y is held by T2)

Transaction T2:

- Locks Y
- Tries to lock X (but X is held by T1)

Result: Both transactions are now stuck, waiting for each other to release their respective locks — creating a **deadlock**.

Deadlock

Deadlock occurs if **all four** of the following conditions hold:

- 1. Mutual Exclusion** — Only one transaction can hold a lock on a data item at any given time.
- 2. Hold and Wait** — Transactions holding resources can request additional resources.
- 3. No Preemption** — A transaction cannot forcibly remove another transaction's lock; only the lock holder can release it.
- 4. Circular Wait** — A circular chain of transactions exists, where each transaction is waiting for a resource held by the next transaction in the chain.

Deadlock Detection Techniques

1.Wait-for Graph (WFG):

1. Nodes represent transactions; directed edges represent waiting relationships.
2. If a cycle is detected in the graph, a deadlock is present.

2.Timeouts:

1. If a transaction waits too long for a resource, it is assumed to be in a deadlock and is forcefully aborted.

Variations of 2PL

(All can guarantee serializability)

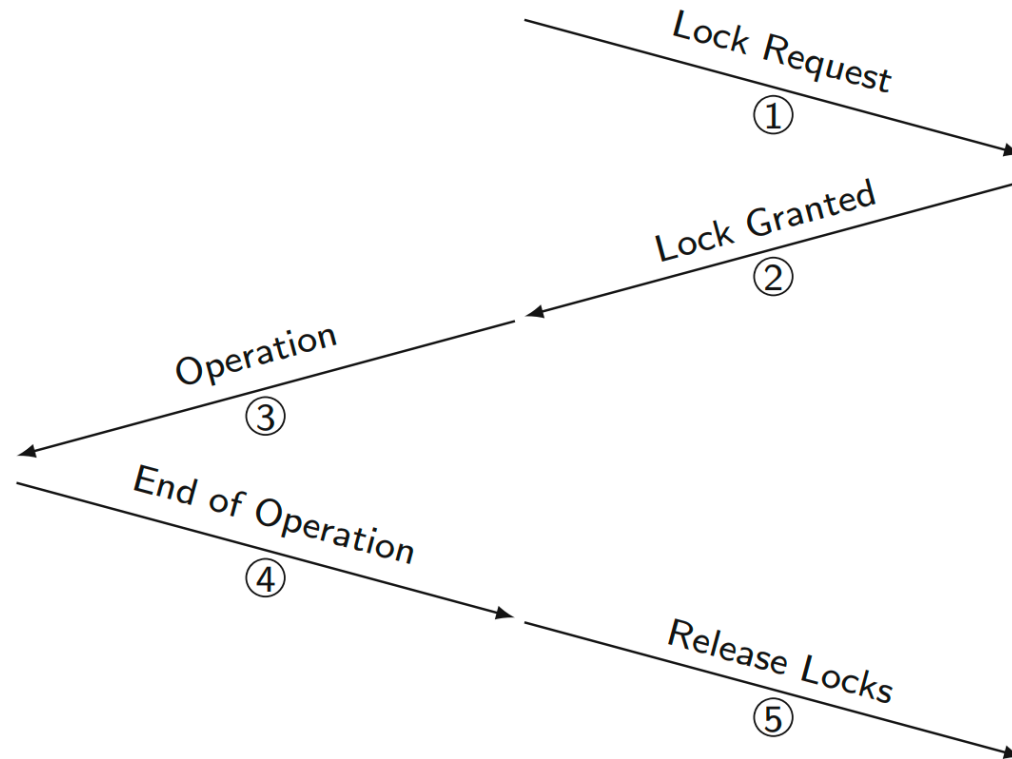
Variation	Key Feature	Deadlock Risk	Best for
Basic 2PL	Locks acquired one-by-one, released one-by-one	Yes	Simple implementation
Conservative 2PL	All locks acquired upfront before any operation	No	Deadlock prevention
Strict 2PL	Write locks held until commit/abort	Yes	Strong recoverability
Rigorous 2PL	All locks (read/write) held until commit/abort	Yes	Maximum recoverability and simplicity in rollback

Distributed 2PL

Data processors at
participating sites

Coordinating TM

Lock Manager



The 2PL algorithm can easily be extended to the distributed DBMS environment by delegating lock management responsibility to a single site. This means that only one of the sites has a lock manager; the transaction managers at the other sites communicate with it to obtain locks.

Durability and Recovery

All-or-nothing

- **All**: When a transaction is submitted to a DBMS, all the operations are completed successfully and their effect is recorded permanently, OR
- **Nothing**: The transaction has no effect on the database or any other transactions

What happens if a failure occurs during a transaction (or in a schedule of transactions)?

- **Failure types** include: *computer failure or system crash, transaction or system errors, local errors or exception conditions, concurrency control enforcement, disk failure, physical problems...*

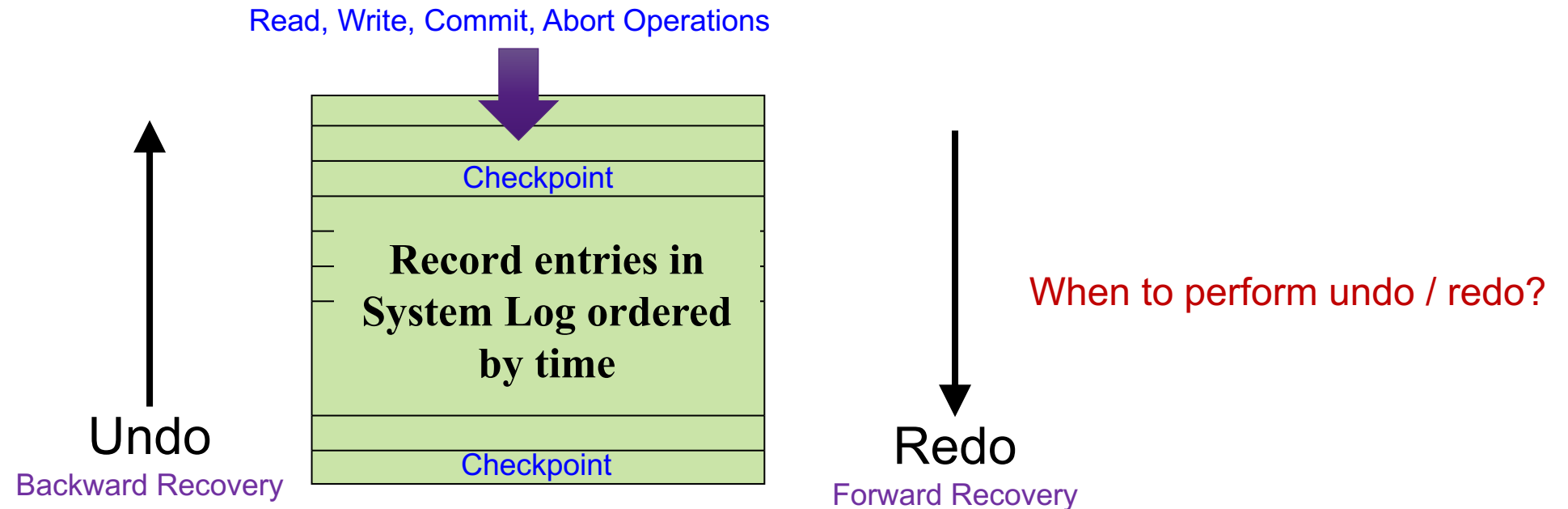
Recoverable Schedule

A transaction in schedule cannot commit until the transactions it reads from are all committed.

System Log

A **journal** that keeps track of all transaction operations on database items

This is to facilitate recovery from failure



UNDO recovered in backward and REDO recovered in forward?

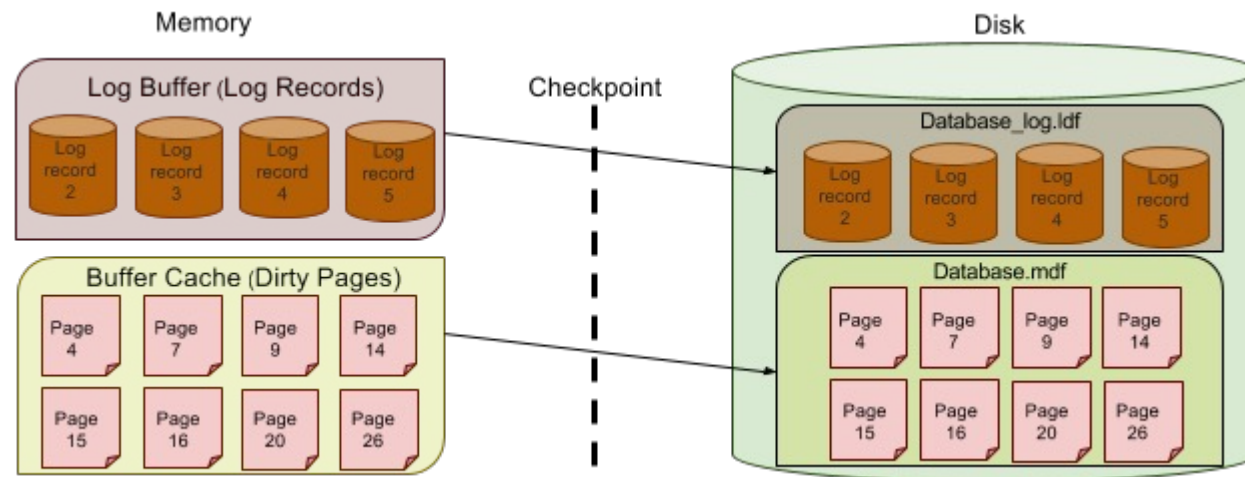
Database state: an important concept

Database State: a snapshot of the database population, with a time stamp. It is a dump of the database with the Log files.

Database log: a timestamped system file to store the changes of a database including the changes of meta data and data records. Database log files can be used for auditing of database performance, system recovery, and security control, etc.

Checkpoint: one of the records in Log file. At the checkpoint, all committed transactions in database buffers will be flushed out (i.e., made permanent on the disk), all active transactions will be force-written out.

Checkpoints are used to reduce the time required for recovery in the event of system failure



Database state: an important concept

```
BEGIN TRANSACTION;  
UPDATE Accounts SET balance = balance - 500 WHERE account_id = 'A';  
UPDATE Accounts SET balance = balance + 500 WHERE account_id = 'B';  
COMMIT;
```

Transaction

```
[LOG] T1 START  
[LOG] T1: UPDATE Accounts SET balance = balance - 500 WHERE account_id = 'A'  
[LOG] T1: UPDATE Accounts SET balance = balance + 500 WHERE account_id = 'B'  
[LOG] T1 COMMIT
```

Log

```
[CHECKPOINT] AT TIME 10:05
```

Checkpoint

All committed transaction data has been safely written to disk before the checkpoint. A checkpoint is an optimization mechanism that marks the starting point for system recovery, avoiding unnecessary log backtracking and improving recovery speed.

Recovery Using the Checkpoint

When a system restarts after a crash, the database engine performs the following steps:

1. Identify the Last Checkpoint: The engine locates the most recent valid checkpoint record in the transaction log.

2. Analyze Phase:

1. Reads forward from the checkpoint to figure out which transactions were in progress.
2. Identifies which database pages were dirty (modified but not yet on disk) at the time of the crash.

3. Redo Phase:

1. Re-applies all changes from committed transactions that occurred after the last checkpoint but had not yet been fully written to disk.
2. Ensures the database reflects all committed operations.

4. Undo Phase:

1. Rolls back any uncommitted transactions identified during the analysis phase.
2. Ensures that the database does not reflect incomplete operations.

Because we start from the last checkpoint (rather than from the very beginning of the log), the amount of work required to redo or undo transactions is much smaller, significantly speeding up recovery.

Write Ahead Logging (WAL)

1. Must **force-write** the log record for an update *before* the corresponding data page is **written** (becomes *dirty*).
2. Must write all log records for a transaction *before commit*.

The first guarantees Atomicity.

The second guarantees Durability.

A **schedule** defines the **order of execution** of operations from multiple transactions.

Recoverable Schedules

A **recoverable schedule** ensures that committed transaction do not need to be rolled back during recovery.

In contrast, a **non-recoverable schedule** may require rolling back a committed transaction, which violates the **Durability** property of the ACID model.

Key Conditions for a Recoverable Schedule

- If one transaction (T2) depends on another transaction (T1), **T2 should only commit after T1 commits.**
- This rule ensures that committed transactions never need to be undone.

Distributed Transaction Management

Distributed database transaction: a set of operations on data that is performed across multiple data repositories based on a shared global database schema. It needs to be coordinated across separate nodes connected by a network.

Distributed database transactions must ALSO satisfy the **ACID property**:

- all distributed operations successfully complete,
- none of the operations are performed at all due to a failure somewhere in the system.

Distributed Transactions

Centralized transaction

- ...is a single process running on a single processor

Distributed transaction

- ...involves multiple **sub-transaction** processes running on multiple processors

Commit in Distributed Databases?

Distributed DBMSs may encounter problems not found in a centralized environment.

- Failure of an **individual site**
- Failure of **communication links**

If **sub-transactions** of a transaction T_x execute at different sites, **all or none** must commit.

Need a **commit protocol** to achieve this.

Recording the effect of database access operations in the **(local) system log is not a sufficient** condition for distributed commit.

Two-Phase Commit (2PC) Protocol

Two-Phase Commit Protocol: **2PC $\neq$ 2PL**

- *2PL $\Rightarrow$ Enforcing Serializability*
- 2PC $\Rightarrow$ Enforcing Atomic transactions (all or nothing)

The Site at which a transaction T_x originates is called **coordinator**

Other Sites which executes the transaction is called **subordinates (participants)**.

Two-Phase Commit (2PC) Protocol

When a **global** transaction T_x wants to commit:

Coordinator sends **Prepare** *msg* (message) to each **subordinate**.

Subordinate writes an **Abort** or **Prepare** log record and then sends a **no** or **yes** *msg* (message) back to coordinator.

Two-Phase Commit (2PC) Protocol

If coordinator gets **unanimous** yes votes:

- The coordinator writes a commit log record and sends commit message to all subordinates

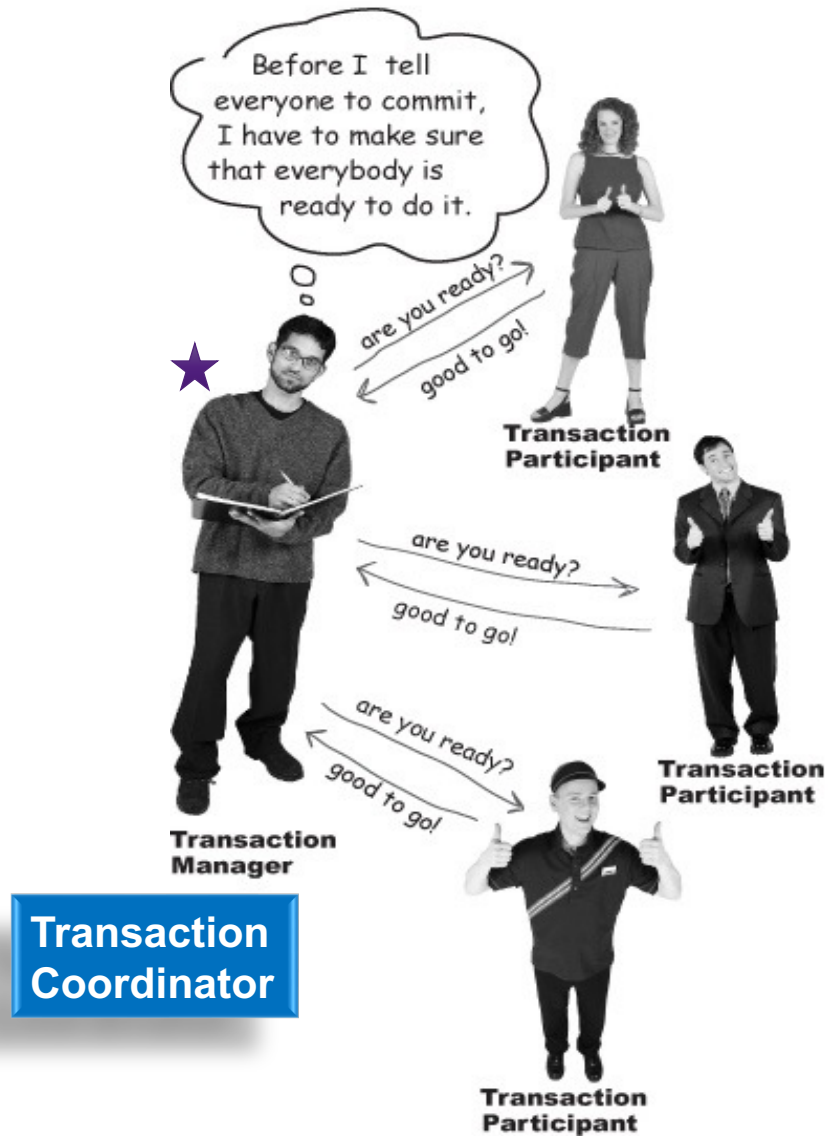
Else

- writes abort log rec, and sends abort message

Subordinates write abort/commit log record based on the message they get, then send **ack** message to coordinator

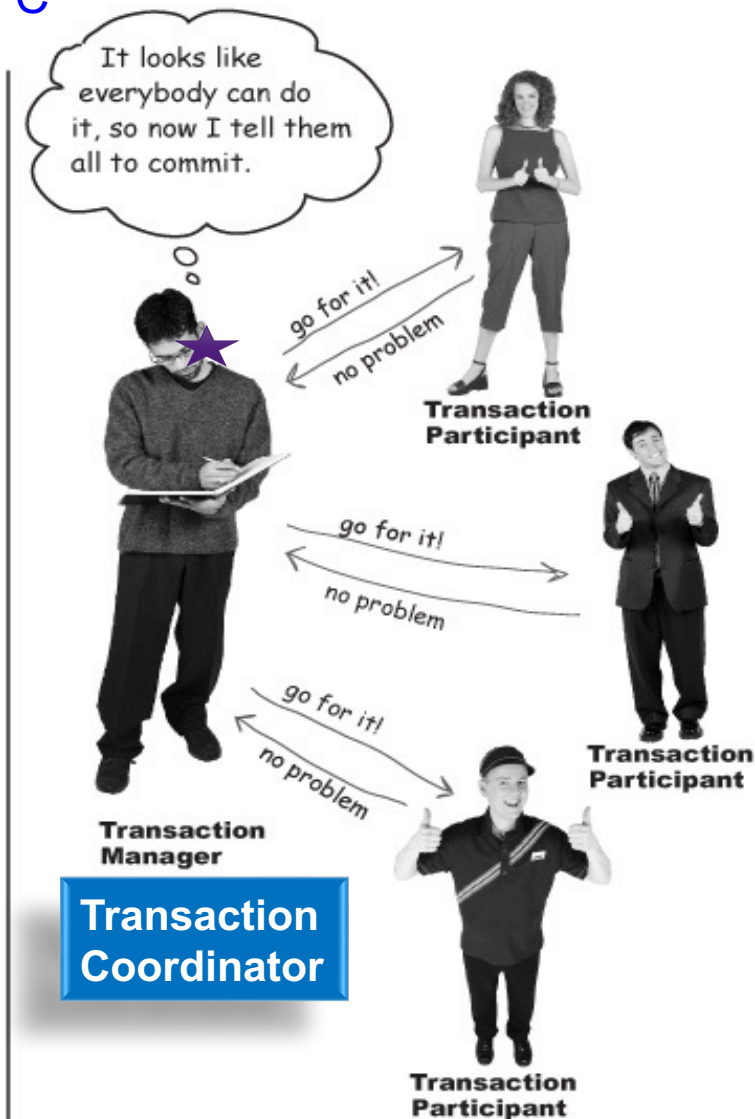
Coordinator writes end log record after getting all **acks**

Phase ONE



2PC

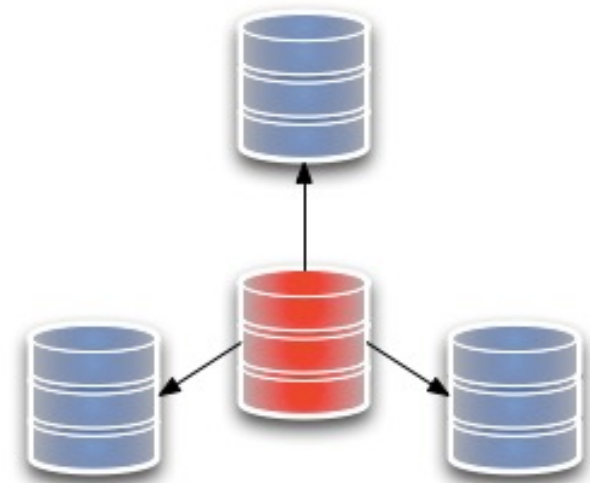
Phase TWO



Maintaining Replicas of Distributed Databases

Assumptions of a DDB:

- A **Global Schema** is available (Data query is transparent)
- The Database is **fragmented** (Data communications are needed)
- Data Records are **replicated** in different locations (Update propagation is needed)
- **Strong Mutual Consistency** of Replicas can be achieved with 2PC – all copies of a data item have the same value at the end of the execution of an update transaction
- **Weak Mutual Consistency** does not require values of replicas of a data item to be identical when an update transaction terminates. What is required is that, if the update activity ceases for some time, the values eventually become identical.



Updating Distributed Data

Synchronous Replication (Eager approach)

- “All copies” of a modified relation (fragment) must be updated before the modifying transaction commits
- Data distribution is made **transparent** to users

Asynchronous Replication (Lazy approach)

- Copies of a modified relation are only **periodically** updated; different copies may get out of sync in the meantime
- Users must be **aware of data distribution**
- Most products follow this approach

Synchronous Replication

Voting ($n > s$) strategy: A transaction must write a majority of copies to modify an object; must read enough copies to be sure of seeing **at least one most recent copy**

- E.g., for 10 (**n**) copies; we write 7 (**s**) copies for an update; then we read 4 (**n-s+1**) copies each time for a request of reading.
- Each copy has a version number
- The copy with the **highest** version number is current
- Not attractive usually, because reads are common

Read-any Write-all ($n = s$) strategy: In this strategy, writes are slower and reads are faster, relative to a voting-based strategy

- A very common approach to support synchronous replication

Cost of Synchronous Replication

Before an update transaction can commit, it must **obtain locks on all copies** to modify

- Sends lock requests to remote sites, and while waiting for the response, **holds on to all other locks!**
- If sites or links fail, transaction cannot commit until they are back up

So the alternative (**asynchronous replication**) is more attractive

Asynchronous Replication

Allows the **modifying transaction to commit** before all copies have been changed (and readers look at just one copy)

- Users must be **aware of which copy they are reading**, and that copies may be out-of-sync (for a short period of time)

Two approaches: **Primary Site** and **Peer-to-Peer** replication

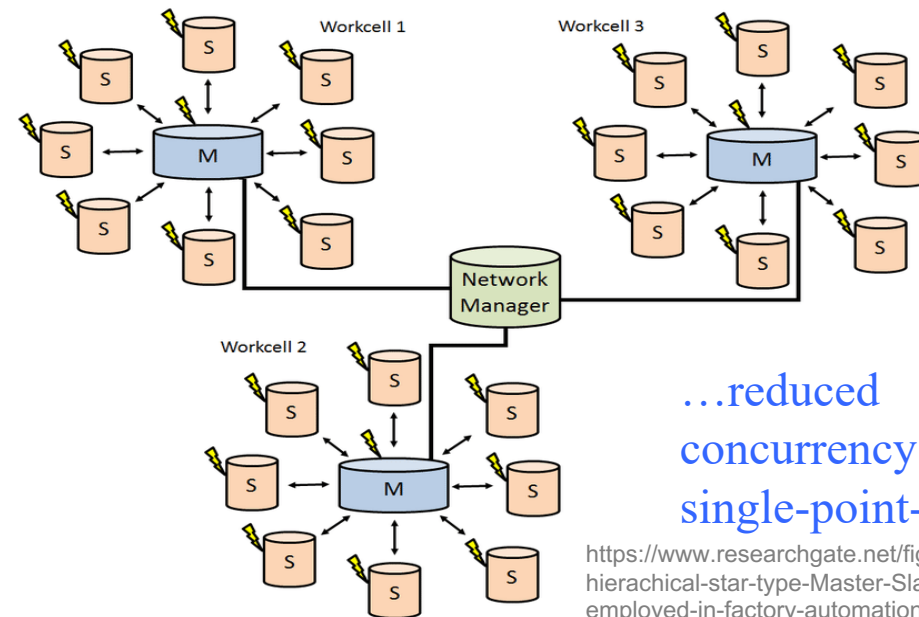
- Both use “**master copies**”.
- The difference lies in how many master copies are “updatable”

Primary Site Replication

Exactly one copy of a relation is designated the **Primary** or **Master copy**. Replicas at other sites cannot be directly updated

- The primary copy is published
- Other sites subscribe to (fragments of) this relation; these are secondary copies

- Main issue: How are the changes to the primary copy propagated to the secondary copies?
 - Done in two steps: (1) 'capture' changes made by committed transactions; (2) 'apply' these changes



...reduced
concurrency and
single-point-of-failure

https://www.researchgate.net/figure/Typical-hierarchical-star-type-Master-Slave-architecture-employed-in-factory-automation_fig1_319199634

Peer-to-Peer Replication

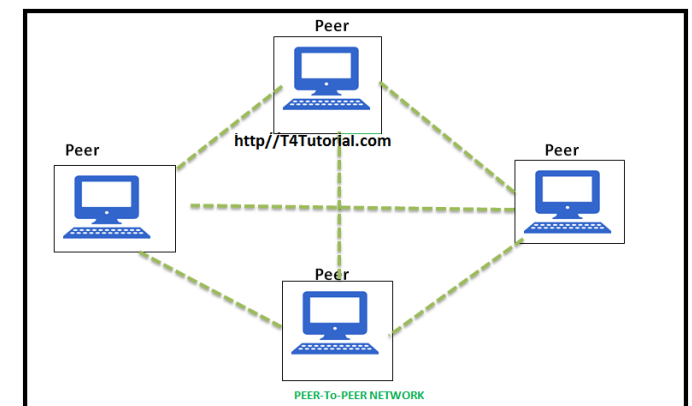
More than one of the copies of an object can be a Master in this approach

Changes to a Master copy must be propagated to other copies somehow

If two Master copies are changed in a conflicting manner, this must be resolved

Best used when conflicts do not arise:

- E.g., Each Master site owns a disjoint fragment
- E.g., Updating rights owned by one master at a time



<https://t4tutorials.com/peer-to-peer-network-characteristics-advantages-disadvantages/>

Review

ACID property is enforced for DDB to handle **concurrent transactions**.

A correct schedule of concurrent transactions is equivalent to **a serial of transactions**.

There may be **conflict operations** in a **non-serial schedule**, which has **interleaved operations** of transactions, i.e., the same data item is read/write by multiple transactions concurrently.

Some **conflict schedules are serializable** and **recoverable**.

2PL protocols are used to guarantee the **serializability** of a schedule.

Deadlock could happen in a serializable schedule.

A distributed transaction can be committed or aborted, by using **2PC** protocol.

There are many important relevant concepts in Distributed TM, such as *Concurrent Transactions, ACID, Interleaving transactions, serializability, Deadlock, WAL, Checkpoint, Rollback, Abort, Commit, Redo, Undo, Synchronous/Asynchronous Replication, 2PL, 2PC*, etc.